PROJECT REPORT

GRID:

AN AUTONOMOUS SCHEDULING SYSTEM

- Course Title: **ICS214 IT WORKSHOP III**
- Student Name: **Keerthana A E**
- Roll Number: **2025BCD0029**
- GitHub Repository: **https://github.com/scrymgeor/Grid_Planner**
- Live Deployment: **https://grid-planner-jmz0.onrender.com/**

TABLE OF CONTENTS

S.No	Title	Page No.
1.	**ABSTRACT**	**2**
2.	**PROBLEM STATEMENT & OBJECTIVES**	**3**
	2.1 Problem Statement	
	2.2 Project Objectives	
3.	**MATHEMATICAL MODELING & METHODOLOGY**	**4**
	3.1 The Urgency Scoring Heuristic	
	3.2 Polymorphic Urgency Scaling	
	3.3 The Greedy Chunking Algorithm	
4.	**SYSTEM ARCHITECTURE & DATABASE DESIGN**	**7**
	4.1 System Block Diagram	
	4.2 Relational Database Schema (SQLite3)	
5.	**TECHNICAL IMPLEMENTATION DETAILS**	**8**
	5.1 Object-Oriented Domain Modeling (OOP & Inheritance)	
	5.2 Client-Server Network Services (SMTP & OTP)	
	5.3 Concurrency & Background Daemon Threading	
	5.4 File Processing (CSV Batch I/O & Text Logging)	
	5.5 In-Memory Data Structures (ActionStack)	
6.	**RESULTS & DEMONSTRATION**	**11**
	6.1 Algorithmic Verification	
	6.2 Cloud Deployment (Render.com)	
	6.3 System Screenshots	
7.	**CONCLUSION & FUTURE SCOPE**	**14**
	7.1 Conclusion	
	7.2 Future Scope	
8.	**REFERENCES**	**15**

1. ABSTRACT

Modern calendar and to-do applications operate as passive storage mediums, requiring users to manually decide the chronology, duration, and urgency of tasks. This results in decision fatigue, miscalculated deadlines, and academic burnout.

This project presents Grid, an autonomous, client-server productivity engine developed in Python and Flask. Grid treats schedule generation as a greedy heuristic optimization problem. Using an in-memory dynamic urgency heuristic, the engine computes time-to-deadline proximity, applies polymorphic priority weights, segments work into burnout-prevention blocks (max 1.5 hours), and maps them into user-configured weekly availability windows.

The system implements relational data persistence using SQLite3, network protocol integration via SMTP, multithreaded daemon monitoring, dynamic data visualization through Matplotlib, bidirectional CSV file I/O, and an in-memory Stack data structure for undo functionality. The user interface utilizes a retro 90s lo-fi desktop aesthetic, providing a focused, responsive user experience deployed both locally and in the cloud.

2. PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

Undergraduate students concurrently manage coursework, lab submissions, exam preparation, and personal habits. Traditional productivity tools fail to solve this because:

1. Lack of Dynamic Urgency: Deadlines approach linearly, but task priority must increase exponentially as the window of opportunity shrinks.
2. Cognitive Overhead: Students spend excessive time organizing tasks rather than executing them.
3. Burnout from Monolithic Scheduling: Students schedule unrealistic 6-hour study sessions without proper segmentation.

2.2 Project Objectives

- Algorithmic Allocation: Develop a deterministic heuristic scheduler that automates time allocation.
- Domain Modeling: Implement strict Object-Oriented Programming (OOP) principles using polymorphism and inheritance to model diverse task types.
- Persistent Architecture: Design a normalized SQLite relational database supporting multi-user isolation and full CRUD operations.
- Network & Security Integration: Implement cryptographic password hashing (SHA-256) and automated SMTP email dispatching for user onboarding and password recovery.
- Workload Analytics: Embed automated data analytics using Matplotlib to generate real-time workload distribution charts.

3. MATHEMATICAL MODELING & METHODOLOGY

3.1 The Urgency Scoring Heuristic

To determine which task claims the earliest available study slot, the engine dynamically evaluates pending tasks at timestamp

$$t_{\text{current}}$$

using the following heuristic formula:

$$\text{Urgency Score} = \frac{\text{Priority} \times 100}{\max(h_{\text{remaining}}, 0.1)}$$

Where:

- $\text{Priority} \in [1, 5]$: User-assigned weight (1 = Low, 5 = Critical).
- $h_{\text{remaining}} = \frac{t_{\text{deadline}} - t_{\text{current}}}{3600}$: Continuous hours remaining until the deadline.
- If $h_{\text{remaining}} \leq 0$, the task is assigned a critical value (99999.0), forcing immediate scheduling.

3.2 Polymorphic Urgency Scaling

Different real-world commitments require different urgency modifiers:

1. **Academic Tasks:** Scaled by course grade weightage (W_{grade}):

$$\text{Score}_{\text{Academic}} = \text{Score}_{\text{Base}} \times \left(1 + \frac{W_{\text{grade}}}{100}\right)$$

2. **Fitness / Routine Habits:** Scaled by active streak days (S_{days}) to incentivize consistency:

$$\text{Score}_{\text{Fitness}} = \text{Score}_{\text{Base}} + (S_{\text{days}} \times 5)$$

3. **Work / Milestone Tasks:** Scaled by a baseline deliverable factor ($1.25\times$).

3.3 The Greedy Chunking Algorithm

1. Retrieve all pending tasks (T) and upcoming availability windows (W) within a 7-day lookahead.
2. Filter out completed tasks and sort Tin descending order of their computed Urgency Score:

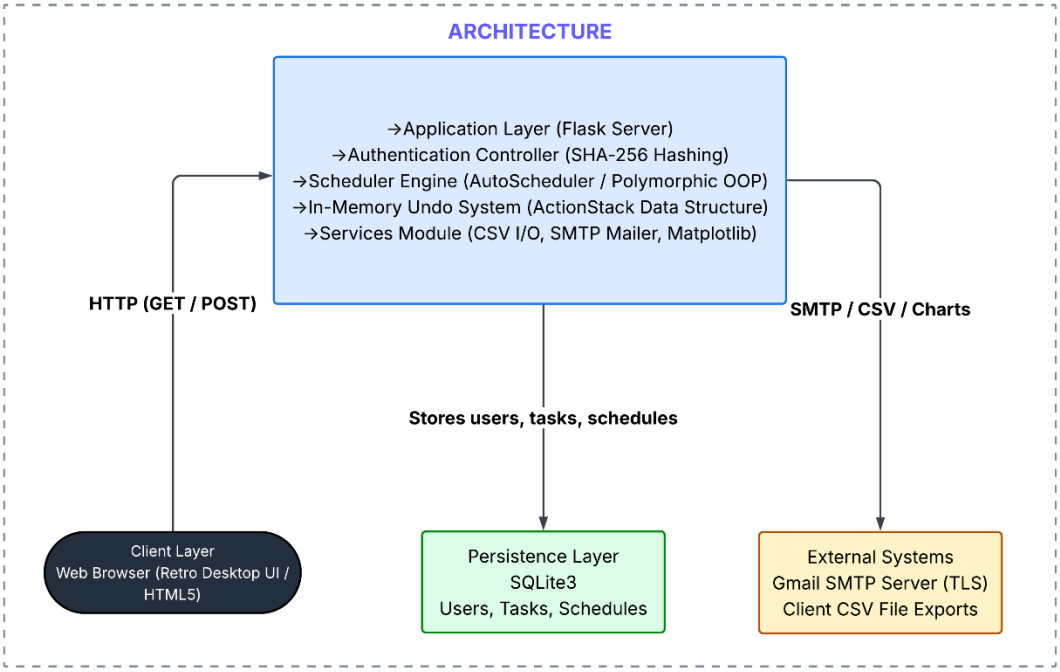
$$\text{Queue} \leftarrow \text{sort}(T, \text{key}, = \lambda t{:}\text{Urgency}(t)\text{reverse} = \text{True})$$

3. Iterate through chronological time slots $w \in W$:
    - While slot $w$has remaining capacity and tasks exist in the queue:
        - Extract the highest-urgency task t_{top}.
        - Compute session duration:

$$\text{Duration} = \min\left(w_{\text{remaining}}, t_{\text{remaining}}, 1.5 \text{ hours}\right)$$

        - Commit the allocated block into the database.
        - Decrement $w_{\text{remaining}}$ and $t_{\text{remaining}}$.
        - If $t_{\text{remaining}} \leq 0$, remove t_{top} from the queue.

4. SYSTEM ARCHITECTURE & DATABASE DESIGN



4.1 Database Schema (SQLite3)

- users Table: Stores id, username, email, password_hash, and temporary reset_token.
- tasks Table: Stores id, user_id (FK), title, subject, task_type, deadline, estimated_hours, priority, specific_meta, color_tag, and is_completed.
- free_slots Table: Stores recurring availability windows: user_id (FK), day_of_week (0–6), start_hour (0–23), and duration_hours.
- schedule Table: Contains the output of the scheduling algorithm: user_id (FK), task_id (FK), task_title, session_date, start_time, and end_time.
- habits Table: Tracks recurring daily habits: user_id (FK), title, streak, and last_completed timestamp.

5.TECHNICAL IMPLEMENTATION

5.1 Object-Oriented Programming (OOP)

- Encapsulation: The internal state of tasks and time slots are encapsulated within domain objects. Passwords are never stored in plain text.
- Inheritance & Polymorphism: Subclasses (AcademicTask, FitnessHabitTask, WorkProjectTask) inherit from the base Task class and override .get_urgency_score().
- Data Structures: A custom ActionStack class implements a Last-In, First-Out (LIFO) stack with .push() and .pop() methods to provide real-time undo functionality.

5.2 Network Programming (SMTP)

- Implemented using Python's native smtplib and email.mime modules.
- Establishes TLS-encrypted socket connections to Gmail's SMTP relay (Port 587/465).
- Dispatches welcome digests upon account registration and dynamic 6-digit OTP codes for password recovery. Includes an offline simulation fallback for restricted network environments.

5.3 Concurrency & Multithreading

- Uses Python's native threading.Thread with daemon=True to execute a continuous background monitor.
- Polls system states and logs events to system_activity.log every 60 seconds without blocking user requests or stalling Flask's main execution loop.

5.4 File Processing & Data Analytics

- CSV Processing: Uses Python’s csv module for two-way operations: csv.DictReader for batch importing semester tasks, and csv.writer for exporting the generated schedule to a spreadsheet.
- Matplotlib Visualization: Generates server-side categorical bar charts showing workload distribution, rendered in headless mode (Agg backend) and embedded dynamically in the interface.

6. RESULTS

6.1 Algorithmic Verification

Testing demonstrated that given identical target deadlines, an academic assignment with a 25% weightage was prioritized over a general errand. When an urgent task was introduced with a tight deadline, the system automatically fragmented and rescheduled subsequent tasks without manual user intervention.

6.2 Cloud Deployment & Portability

The project was deployed live on Render.com using the Gunicorn WSGI production runner. Testing verified full functional parity between the local offline-first environment and the live cloud deployment.

6.3

Fig 6.3.1. Login Page



Fig 6.3.2.a. Dashboard

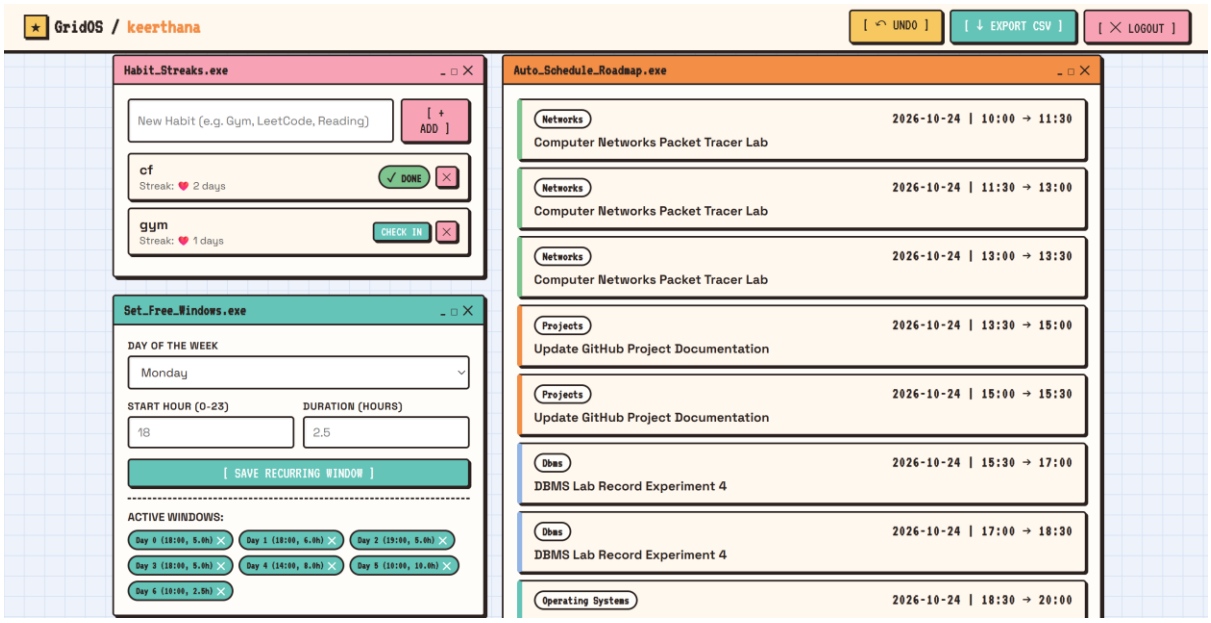


Fig 6.3.2.b. Dashboard

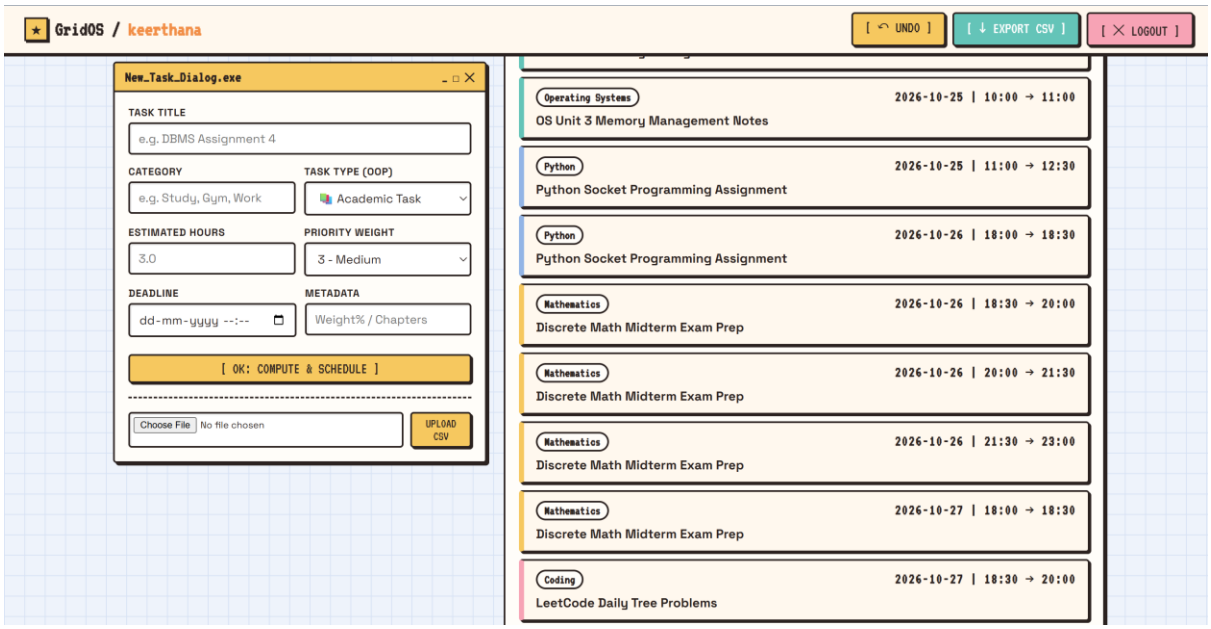


Fig 6.3.3.Matplotlib Analytics Chart

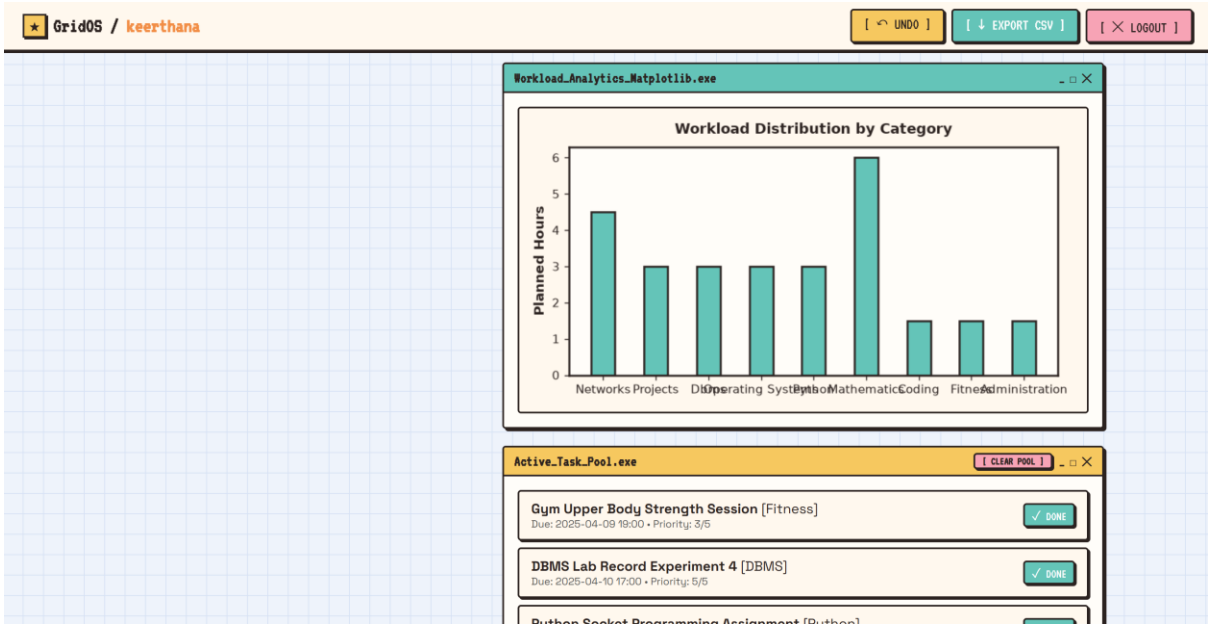


Fig 6.3.4. Habit Tracker showing active daily streak

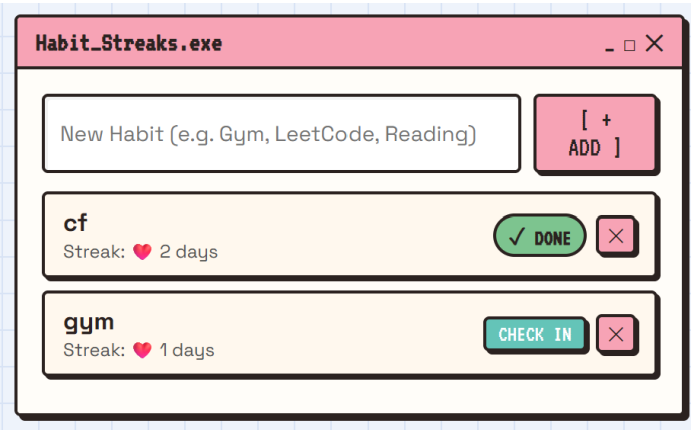
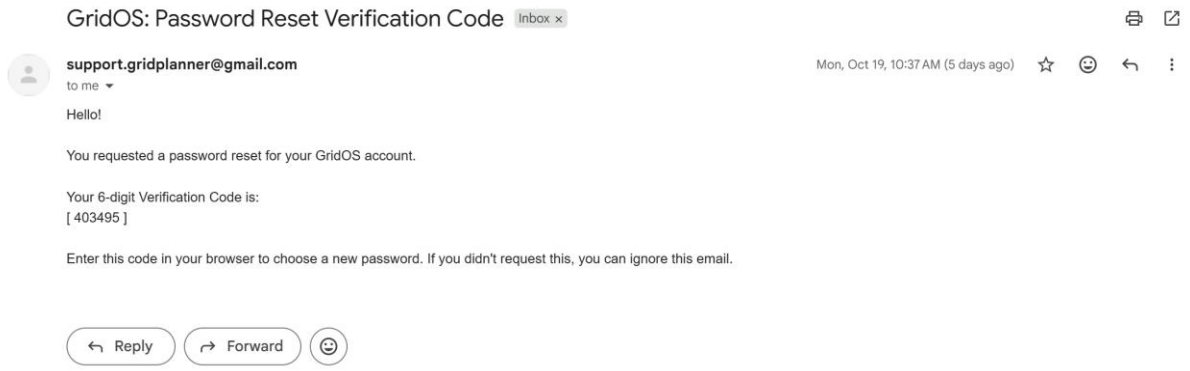


Fig 6.3.5. Password Recovery OTP received in Gmail




Password_Recovery.exe
Reset Password
Enter your email to receive a 6-digit verification code.
REGISTERED EMAIL
student@university.edu
[DISPATCH RESET CODE]
← Back to Login

7. CONCLUSION & FUTURE SCOPE

7.1 Conclusion

Grid successfully bridges the gap between passive calendar tools and complex project management software. By integrating greedy heuristic scheduling, Object-Oriented design patterns, relational SQLite storage, multithreaded daemon monitoring, and SMTP network protocols, the system fulfills all pedagogical requirements of the Semester 3 Python curriculum while providing real-world practical utility.

7.2 Future Scope

- Natural Language Processing (NLP): Ingest raw syllabus documents using text extraction to auto-detect deadlines.
- Calendar Interoperability: Support two-way synchronization with standard .ics (iCalendar) formats.
- Adaptive Velocity Tracking: Dynamically update task duration estimates based on how fast a student completes past assignments.

REFERENCES

1. Python Software Foundation, Python 3.11 Documentation, docs.python.org.
2. Grinberg, M., Flask Web Development: Developing Web Applications with Python, O'Reilly Media.
3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C., Introduction to Algorithms (Greedy Algorithms), MIT Press.
4. Hunter, J. D., Matplotlib: A 2D Graphics Environment, Computing in Science & Engineering.